E-Vehicle Charging Management System for a Smart University CampusMembers

VANGIMALLA BALA SIVA PRASAD REDDY (192424316)

MOHAMED IMRAAN S (192424001)

ALLEN GABRIEL M (192421089)

Pseudo code:

1. Energy Requirement Calculation

```
ALGORITHM CalculateRequiredEnergy(batteryCapacityKwh, currentPercent,
targetPercent)
INPUT:
   batteryCapacityKwh : Real (Total EV battery capacity in kWh)
   currentPercent     : Real (Starting state of charge 0..100)
   targetPercent      : Real (Desired state of charge 0..100)
OUTPUT:
   energyRequiredKwh  : Real (Energy needed to charge in kWh)

BEGIN
   IF currentPercent < 0 OR currentPercent > 100 THEN
      THROW IllegalArgumentException("Invalid current battery percentage")
   END IF
   IF targetPercent <= currentPercent OR targetPercent > 100 THEN
      THROW IllegalArgumentException("Target battery must exceed current battery")
   END IF

   percentageDiff ← (targetPercent - currentPercent) / 100.0
   energyRequiredKwh ← batteryCapacityKwh * percentageDiff

   RETURN ROUND(energyRequiredKwh, 2)
```

END

2. Estimated Charging Time & Cost Calculation

ALGORITHM CalculateChargingTimeAndCost(energyRequiredKwh, chargerPowerKw, tariffRatePerKwh)

INPUT:

energyRequiredKwh : Real (Energy required in kWh)

chargerPowerKw : Real (Charger power rating in kW)

tariffRatePerKwh : Real (Tariff rate from tariffs table in INR/kWh)

OUTPUT:

estimatedHours : Real

estimatedMinutes : Integer

estimatedCost : Real

BEGIN

estimatedHours ← energyRequiredKwh / chargerPowerKw

estimatedMinutes ← CEILING(estimatedHours * 60)

estimatedCost ← ROUND(energyRequiredKwh * tariffRatePerKwh, 2)

RETURN (estimatedHours, estimatedMinutes, estimatedCost)

END

3. Reservation Interval Overlap Conflict Detection

ALGORITHM CheckReservationConflict(pointId, requestedStart, requestedEnd, excludeResId)

INPUT:

pointId : Integer

requestedStart : DateTime

requestedEnd : DateTime

excludeResId : Integer (optional, for update)

OUTPUT:

```
    hasConflict    : Boolean

BEGIN
    SQL ← "SELECT 1 FROM reservations
           WHERE point_id = ?
             AND status IN ('CONFIRMED', 'ACTIVE', 'PENDING')
             AND start_time < ?
             AND end_time > ?"

    EXECUTE PreparedStatement WITH (pointId, requestedEnd, requestedStart)

    IF ResultSet.next() THEN
        RETURN TRUE  // Conflict exists
    ELSE
        RETURN FALSE // No overlap, slot is free
    END IF
END
```

---

4. Campus Electrical Load Ceiling Protection

```text
ALGORITHM ValidateCampusLoad(additionalPowerKw, maxCapacityKw)
INPUT:
    additionalPowerKw : Real (Power rating of candidate charger in kW)
    maxCapacityKw     : Real (Campus threshold, default 100.0 kW)
OUTPUT:
    isAllowed         : Boolean
```

```
BEGIN
   currentActiveLoad ← SELECT COALESCE(SUM(p.charger_power_kw), 0)
                       FROM charging_sessions cs
                       JOIN charging_points p ON cs.point_id = p.point_id
                       WHERE cs.status = 'ACTIVE'

   projectedLoad ← currentActiveLoad + additionalPowerKw

   IF projectedLoad > maxCapacityKw THEN
      RETURN FALSE // Exceeds grid safety threshold
   ELSE
      RETURN TRUE  // Safe to allocate
   END IF
END
```

---

5. Intelligent 5-Factor Charger Recommendation & Ranking

```text
ALGORITHM RecommendBestCharger(vehicle, currentSoC, targetSoC, prefLocation,
reqStart, expDeparture)
INPUT:
   vehicle      : Vehicle (EV entity with batteryCapacity and connectorType)
   currentSoC   : Real (Current %)
   targetSoC    : Real (Target %)
   prefLocation : String (Preferred campus location)
   reqStart     : DateTime (Requested start)
   expDeparture : DateTime (Expected departure)
```

```
OUTPUT:
    bestCharger  : CandidateScoreDto or NULL (Virtual Queue recommended)

BEGIN
    energyRequired ← CalculateRequiredEnergy(vehicle.capacity, currentSoC, targetSoC)
    compatiblePoints ← SELECT * FROM charging_points WHERE connector_type =
vehicle.connectorType AND status != 'INACTIVE'
    currentLoad ← GetCurrentActiveCampusLoad()
    candidatesList ← EMPTY_LIST

    FOR EACH point IN compatiblePoints DO
        (durationHours, durationMinutes, cost) ←
CalculateChargingTimeAndCost(energyRequired, point.power, point.tariff)
        compTime ← reqStart + durationMinutes
        projectedLoad ← currentLoad + point.power

        eligible ← TRUE

        // Check 1: Maintenance
        IF point.status == 'MAINTENANCE' THEN
            eligible ← FALSE
        END IF

        // Check 2: Schedule Overlap
        IF CheckReservationConflict(point.id, reqStart, compTime, NULL) THEN
            eligible ← FALSE
        END IF

        // Check 3: Campus Load Constraint (<= 100 kW)
        IF projectedLoad > 100.0 THEN
```

```
    eligible ← FALSE
END IF

// 5-Factor Scoring (Total 100)
score ← 0.0

// 1. Availability Suitability (30 pts)
IF eligible AND point.status == 'AVAILABLE' THEN
    score ← score + 30.0
ELSE IF eligible AND point.status == 'OCCUPIED' THEN
    score ← score + 15.0
END IF

// 2. Waiting Time Suitability (25 pts)
waitMins ← (point.status == 'AVAILABLE') ? 0 : 15
score ← score + MAX(0, 25.0 - (waitMins * 0.5))

// 3. Campus Load Efficiency (20 pts)
IF eligible AND projectedLoad <= 80.0 THEN
    score ← score + 20.0
ELSE IF eligible AND projectedLoad <= 100.0 THEN
    score ← score + 14.0
END IF

// 4. Preferred Location Suitability (15 pts)
IF point.campusLocation == prefLocation THEN
    score ← score + 15.0
ELSE
    score ← score + 6.0
```

```
        END IF

        // 5. Completion Before Departure (10 pts)
        IF eligible AND (expDeparture == NULL OR compTime <= expDeparture) THEN
            score ← score + 10.0
        ELSE
            score ← score + 2.0
        END IF

        candidateDto ← CreateCandidateDto(point, score, eligible, energyRequired,
durationMinutes, cost, compTime, projectedLoad)
        candidatesList.ADD(candidateDto)
    END FOR

    SORT candidatesList BY totalScore DESCENDING

    bestCandidate ← FIRST eligible candidate IN candidatesList
    IF bestCandidate != NULL THEN
        RETURN bestCandidate
    ELSE
        RETURN NULL // Signal Virtual Queue Prompt
    END IF
END
```

---

6. Virtual Queue Priority Score & Automatic Promotion

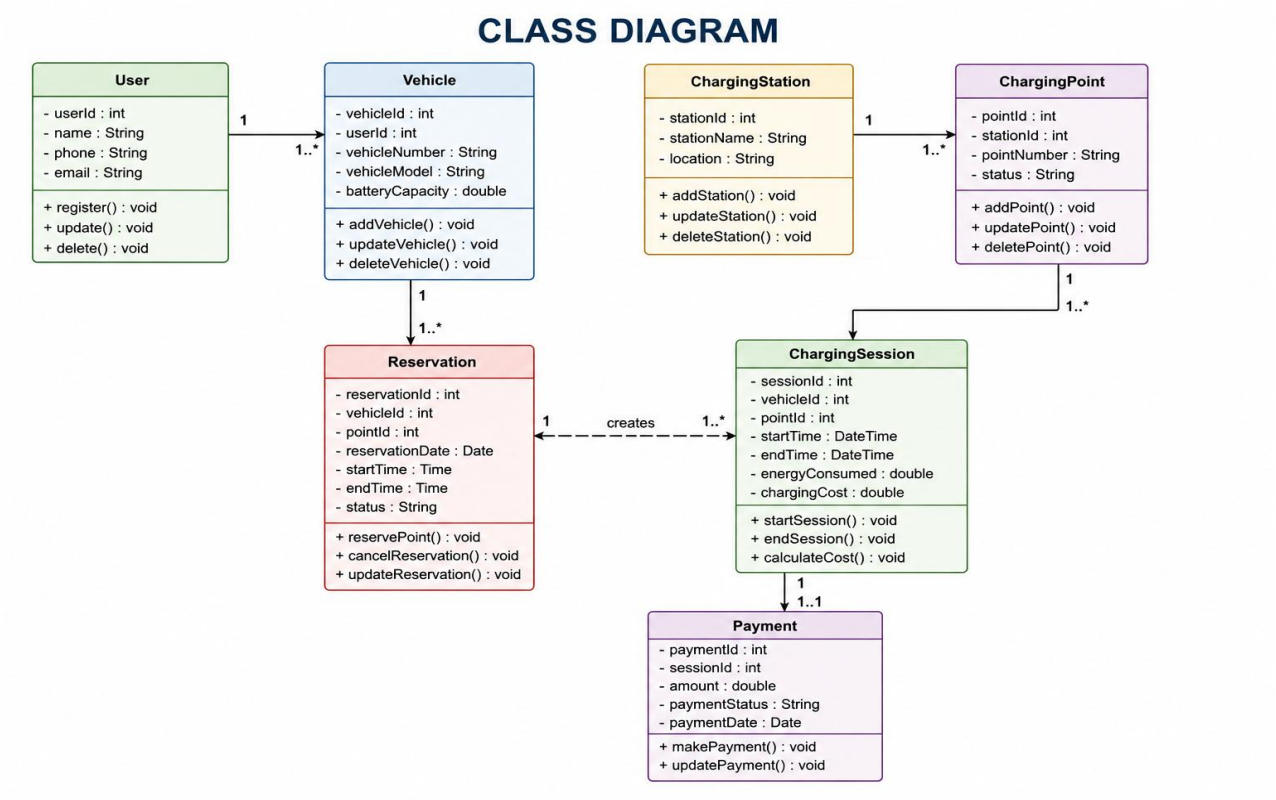
```text
ALGORITHM CalculateQueuePriorityScore(batteryPercent, departureTime, createdTime)
```

```
INPUT:
    batteryPercent : Real (Current battery %)
    departureTime  : DateTime (Departure constraint)
    createdTime    : DateTime (Queue entry insertion timestamp)
OUTPUT:
    priorityScore  : Real

BEGIN
    // 1. Battery Urgency (10..40 pts)
    IF batteryPercent <= 15.0 THEN
        bScore ← 40.0
    ELSE IF batteryPercent <= 30.0 THEN
        bScore ← 30.0
    ELSE IF batteryPercent <= 50.0 THEN
        bScore ← 20.0
    ELSE
        bScore ← 10.0
    END IF

    // 2. Departure Urgency (10..40 pts)
    hrsLeft ← DurationBetween(NOW(), departureTime).toHours()
    IF hrsLeft < 1 THEN
        dScore ← 40.0
    ELSE IF hrsLeft <= 2 THEN
        dScore ← 30.0
    ELSE IF hrsLeft <= 4 THEN
        dScore ← 20.0
    ELSE
        dScore ← 10.0
```

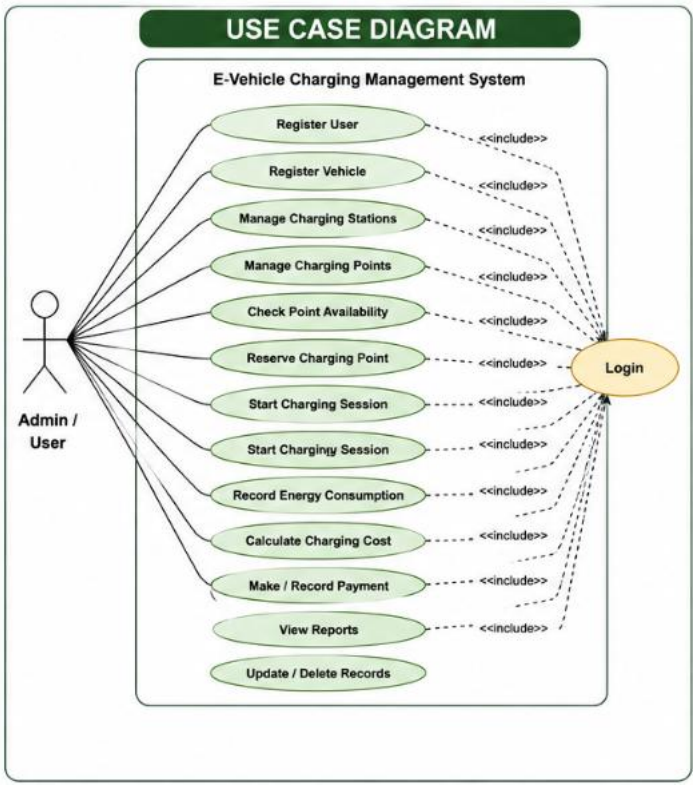
```
    END IF

    // 3. Waiting Duration (+1 point per 5 minutes)
    waitMins ← DurationBetween(createdTime, NOW()).toMinutes()
    wScore ← MAX(0, waitMins / 5.0)

    RETURN ROUND(bScore + dScore + wScore, 1)
END

ALGORITHM PromoteEligibleQueuedVehicles(availablePointId)
INPUT:
    availablePointId : Integer (Newly freed charging point)
BEGIN
    point ← FindPointById(availablePointId)
    IF point.status != 'AVAILABLE' THEN RETURN

    waitingQueue ← SELECT * FROM queue_entries WHERE status = 'WAITING' ORDER
BY priority_score DESC

    FOR EACH entry IN waitingQueue DO
        IF entry.connectorType == point.connectorType THEN
            IF ValidateCampusLoad(point.power, 100.0) THEN
                UPDATE queue_entries SET status = 'PROMOTED' WHERE queue_id = entry.id
                UPDATE charging_points SET status = 'RESERVED' WHERE point_id = point.id
                INSERT INTO reservations (user_id, vehicle_id, point_id, start_time, end_time,
status)
                VALUES (entry.userId, entry.vehicleId, point.id, NOW(), NOW() + 60 mins,
'CONFIRMED')
                BREAK
            END IF
        END IF
```

END FOR

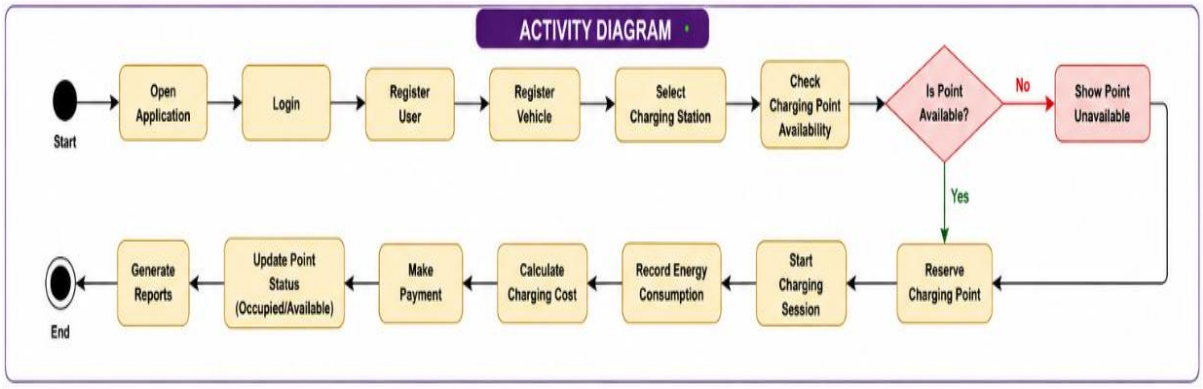
END

```

## Class Diagram:


```

Usecase Diagram:



Activity Diagram:



Java Codes:

```
package com.smartcharge;


import java.sql.*;
import java.time.LocalDateTime;


public class SmartChargeCampus {
```

```
private static final String DB_URL =
        "jdbc:mysql://localhost:3306/smartcharge_campus";

private static final String DB_USER = "root";

// Keep the real password outside source code in the actual project.
private static final String DB_PASSWORD = "YOUR_MYSQL_PASSWORD";

private static final double MAX_CAMPUS_LOAD = 100.0;

// ------------------------------------------------------
// DATABASE CONNECTION
// ------------------------------------------------------

public static Connection getConnection() throws SQLException {

    return DriverManager.getConnection(
            DB_URL,
            DB_USER,
            DB_PASSWORD
    );
}

// ------------------------------------------------------
// 1. STATEMENT DEMONSTRATION
// ------------------------------------------------------

public static void displayChargingStations() {
```

```
String sql = "SELECT * FROM charging_stations";

try (
        Connection conn = getConnection();
        Statement stmt = conn.createStatement();
        ResultSet rs = stmt.executeQuery(sql)
) {

    System.out.println("CHARGING STATIONS");

    while (rs.next()) {

        int stationId =
                rs.getInt("station_id");

        String stationName =
                rs.getString("station_name");

        System.out.println(
                stationId + " - " + stationName
        );
    }

} catch (SQLException e) {

    System.out.println(
            "Database Error: " + e.getMessage()
    );
```

```
    }
}

// ---------------------------------------------------------
// 2. PREPAREDSTATEMENT - VEHICLE REGISTRATION
// ---------------------------------------------------------

public static void registerVehicle(
        int userId,
        String registrationNumber,
        String vehicleType,
        String manufacturer,
        String model,
        double batteryCapacity,
        String connectorType) {

    String sql =
            "INSERT INTO vehicles " +
            "(user_id, registration_number, vehicle_type, " +
            "manufacturer, model, battery_capacity_kwh, connector_type) " +
            "VALUES (?, ?, ?, ?, ?, ?, ?)";

    try (
            Connection conn = getConnection();
            PreparedStatement ps =
                    conn.prepareStatement(sql)
    ) {

        ps.setInt(1, userId);
```

```
            ps.setString(2, registrationNumber);
            ps.setString(3, vehicleType);
            ps.setString(4, manufacturer);
            ps.setString(5, model);
            ps.setDouble(6, batteryCapacity);
            ps.setString(7, connectorType);

            int rows = ps.executeUpdate();

            if (rows > 0) {

                System.out.println(
                        "Vehicle registered successfully."
                );
            }

        } catch (SQLException e) {

            System.out.println(
                    "Vehicle Registration Error: "
                            + e.getMessage()
            );
        }
    }

    // ------------------------------------------------------
    // 3. REQUIRED ENERGY CALCULATION
    // ------------------------------------------------------
```

```
public static double calculateRequiredEnergy(
        double batteryCapacity,
        double currentBattery,
        double targetBattery) {

    if (currentBattery < 0 ||
            targetBattery > 100 ||
            targetBattery <= currentBattery) {

        throw new IllegalArgumentException(
                "Invalid battery percentage."
        );
    }

    return batteryCapacity *
            (targetBattery - currentBattery)
            / 100.0;
}

// ------------------------------------------------------
// 4. CHARGING DURATION
// ------------------------------------------------------

public static double calculateChargingDuration(
        double requiredEnergy,
        double chargerPower) {

    if (chargerPower <= 0) {
```

```
            throw new IllegalArgumentException(
                    "Charger power must be positive."
            );
        }

        return requiredEnergy / chargerPower;
    }

    // ----------------------------------------------------
    // 5. ESTIMATED CHARGING COST
    // ----------------------------------------------------

    public static double calculateCost(
            double energy,
            double tariffPerKwh) {

        return energy * tariffPerKwh;
    }

    // ----------------------------------------------------
    // 6. REAL-TIME CAMPUS LOAD
    // ----------------------------------------------------

    public static double getCurrentCampusLoad() {

        String sql =
                "SELECT COALESCE(SUM(cp.power_kw),0) AS current_load " +
                "FROM charging_sessions cs " +
                "JOIN charging_points cp " +
```

```
            "ON cs.charging_point_id = cp.point_id " +
            "WHERE cs.status = 'ACTIVE'";

    try (
            Connection conn = getConnection();
            PreparedStatement ps =
                    conn.prepareStatement(sql);
            ResultSet rs = ps.executeQuery()
    ) {

        if (rs.next()) {

            return rs.getDouble("current_load");
        }

    } catch (SQLException e) {

        System.out.println(
                "Load Calculation Error: "
                        + e.getMessage()
        );
    }

    return 0;
}

// ------------------------------------------------------
// 7. 100 KW CAMPUS LOAD PROTECTION
// ------------------------------------------------------
```

```
public static boolean canStartCharging(
        double chargerPower) {

    double currentLoad =
            getCurrentCampusLoad();

    double projectedLoad =
            currentLoad + chargerPower;

    System.out.println(
            "Current Campus Load: "
                    + currentLoad + " kW"
    );

    System.out.println(
            "Projected Campus Load: "
                    + projectedLoad + " kW"
    );

    return projectedLoad
            <= MAX_CAMPUS_LOAD;
}

// ---------------------------------------------------------
// 8. 5-FACTOR RECOMMENDATION SCORE
// ---------------------------------------------------------

public static int calculateRecommendationScore(
```

```
    boolean connectorCompatible,
    boolean sufficientEnergy,
    double projectedLoad,
    boolean scheduleAvailable,
    double currentBattery) {

int score = 0;

// Factor 1 - Connector Compatibility
if (connectorCompatible) {
    score += 25;
}

// Factor 2 - Energy Requirement
if (sufficientEnergy) {
    score += 20;
}

// Factor 3 - Campus Load
if (projectedLoad <= 60) {

    score += 20;

} else if (projectedLoad <= 80) {

    score += 15;

} else if (projectedLoad <= 100) {
```

```
        score += 5;
    }

    // Factor 4 - Schedule Availability
    if (scheduleAvailable) {
        score += 20;
    }

    // Factor 5 - Battery Urgency
    if (currentBattery <= 15) {

        score += 15;

    } else if (currentBattery <= 30) {

        score += 12;

    } else if (currentBattery <= 50) {

        score += 8;

    } else {

        score += 5;
    }

    return Math.min(score, 100);
}
```

```
// ------------------------------------------------------
// 9. RESERVATION CONFLICT CHECK
// ------------------------------------------------------

public static boolean hasReservationConflict(
        int pointId,
        Timestamp startTime,
        Timestamp endTime) {

    String sql =
            "SELECT COUNT(*) AS conflict_count " +
            "FROM reservations " +
            "WHERE charging_point_id = ? " +
            "AND status NOT IN ('CANCELLED','COMPLETED') " +
            "AND ? < end_time " +
            "AND ? > start_time";

    try (
            Connection conn = getConnection();
            PreparedStatement ps =
                    conn.prepareStatement(sql)
    ) {

        ps.setInt(1, pointId);
        ps.setTimestamp(2, startTime);
        ps.setTimestamp(3, endTime);

        try (ResultSet rs = ps.executeQuery()) {
```

```
            if (rs.next()) {

                return rs.getInt(
                        "conflict_count"
                ) > 0;
            }
        }

    } catch (SQLException e) {

        System.out.println(
                "Reservation Check Error: "
                        + e.getMessage()
        );
    }

    return true;
}

// ----------------------------------------------------------
// 10. VIRTUAL QUEUE PRIORITY
// ----------------------------------------------------------

public static int calculateQueuePriority(
        double batteryPercentage,
        double hoursUntilDeparture,
        int waitingMinutes) {

    int batteryUrgency;
```

```
int departureUrgency;

// Battery urgency
if (batteryPercentage <= 15) {

    batteryUrgency = 40;

} else if (batteryPercentage <= 30) {

    batteryUrgency = 30;

} else if (batteryPercentage <= 50) {

    batteryUrgency = 20;

} else {

    batteryUrgency = 10;
}

// Departure urgency
if (hoursUntilDeparture < 1) {

    departureUrgency = 40;

} else if (hoursUntilDeparture <= 2) {

    departureUrgency = 30;
```

```
    } else if (hoursUntilDeparture <= 4) {

        departureUrgency = 20;

    } else {

        departureUrgency = 10;
    }

    // 1 point every 15 minutes
    int waitingScore =
            waitingMinutes / 15;

    return batteryUrgency
            + departureUrgency
            + waitingScore;
}

// ---------------------------------------------------------
// 11. ADD VEHICLE TO VIRTUAL QUEUE
// ---------------------------------------------------------

public static void addToQueue(
        int userId,
        int vehicleId,
        String preferredLocation,
        double currentBattery,
        double targetBattery,
        double hoursUntilDeparture,
```

```
    int waitingMinutes) {

int priority =
        calculateQueuePriority(
                currentBattery,
                hoursUntilDeparture,
                waitingMinutes
        );

String sql =
        "INSERT INTO queue_entries " +
        "(user_id, vehicle_id, preferred_location, " +
        "current_battery_percent, target_battery_percent, " +
        "priority_score, status) " +
        "VALUES (?, ?, ?, ?, ?, ?, 'WAITING')";

try (
        Connection conn = getConnection();
        PreparedStatement ps =
                conn.prepareStatement(sql)
) {

    ps.setInt(1, userId);
    ps.setInt(2, vehicleId);
    ps.setString(3, preferredLocation);
    ps.setDouble(4, currentBattery);
    ps.setDouble(5, targetBattery);
    ps.setInt(6, priority);
```

```
            ps.executeUpdate();

            System.out.println(
                    "Vehicle added to virtual queue."
            );

            System.out.println(
                    "Priority Score = " + priority
            );

        } catch (SQLException e) {

            System.out.println(
                    "Queue Error: "
                            + e.getMessage()
            );
        }
    }

    // ------------------------------------------------------
    // 12. CHARGING SESSION BILLING
    // ------------------------------------------------------

    public static double calculateSessionBill(
            double chargerPower,
            double durationHours,
            double tariffPerKwh) {

        double energyDelivered =
```

```
            chargerPower * durationHours;

    return energyDelivered *
            tariffPerKwh;
}

// ------------------------------------------------------
// 13. CALLABLESTATEMENT
// ------------------------------------------------------

public static void displayStationUtilization() {

    String sql =
            "{CALL GetStationUtilization()}";

    try (
            Connection conn = getConnection();
            CallableStatement cs =
                    conn.prepareCall(sql);
            ResultSet rs = cs.executeQuery()
    ) {

        System.out.println(
                "STATION UTILIZATION REPORT"
        );

        while (rs.next()) {

            System.out.println(
```

```
                rs.getString(1)
                    + " | "
                    + rs.getString(2)
            );
        }

    } catch (SQLException e) {

        System.out.println(
            "Stored Procedure Error: "
                + e.getMessage()
        );
    }
}

// ----------------------------------------------------
// MAIN METHOD
// ----------------------------------------------------

public static void main(String[] args) {

    System.out.println(
        "==================================="
    );

    System.out.println(
        "SMARTCHARGE CAMPUS"
    );
```

```
System.out.println(
        "Intelligent Load-Aware EV Charging"
);

System.out.println(
        "================================="
);

// Statement demonstration
displayChargingStations();

// Example vehicle data
double batteryCapacity = 40.5;
double currentBattery = 20;
double targetBattery = 80;

// Required energy
double requiredEnergy =
        calculateRequiredEnergy(
                batteryCapacity,
                currentBattery,
                targetBattery
        );

System.out.println(
        "\nRequired Energy = "
                + requiredEnergy + " kWh"
);
```

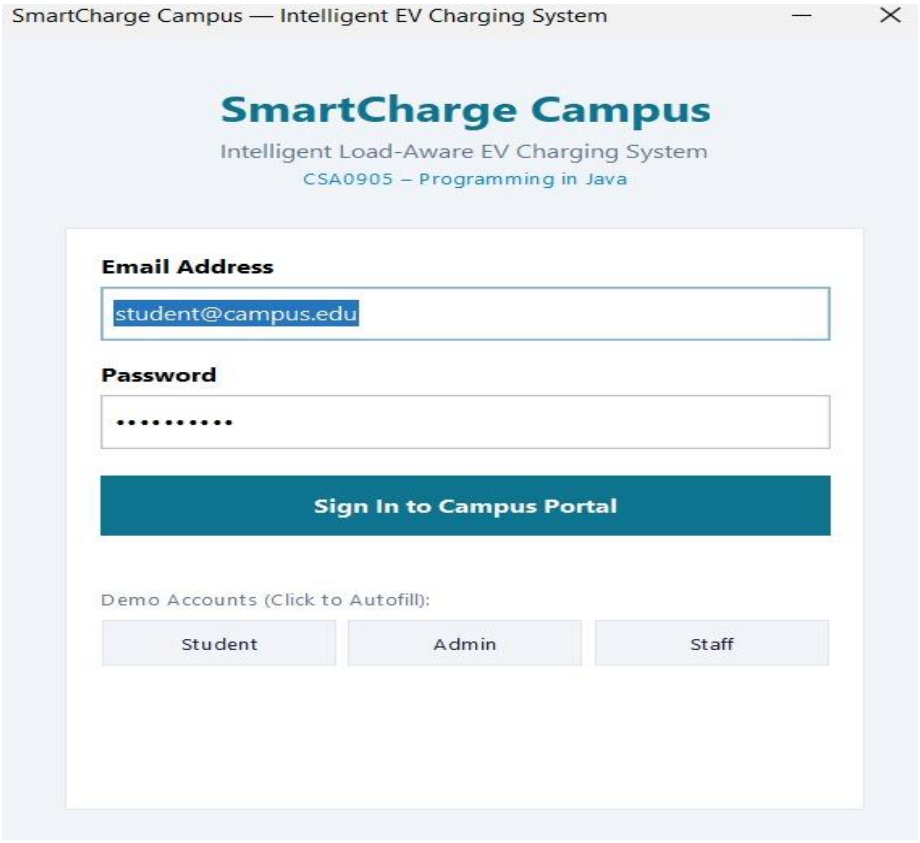
```
// Example charger
double chargerPower = 22.0;

double duration =
        calculateChargingDuration(
                requiredEnergy,
                chargerPower
        );

System.out.println(
        "Estimated Duration = "
                + duration + " hours"
);

double tariff = 7.0;

double estimatedCost =
        calculateCost(
                requiredEnergy,
                tariff
        );

System.out.println(
        "Estimated Cost = ₹"
                + estimatedCost
);

// Campus load checking
if (canStartCharging(chargerPower)) {
```

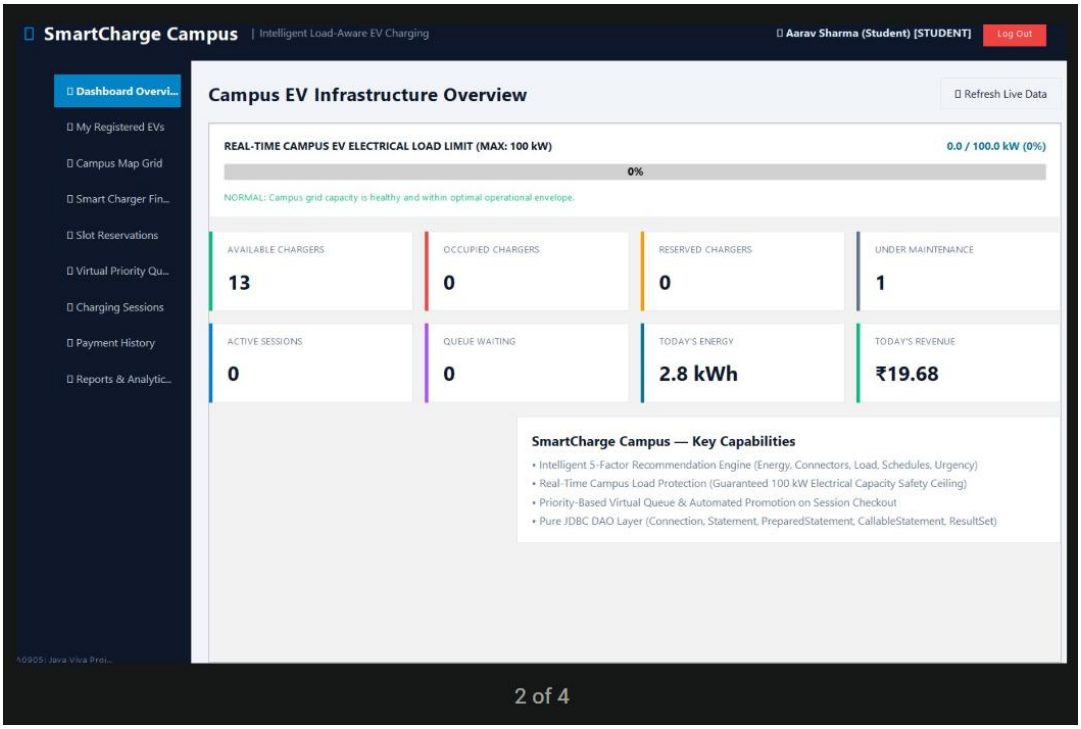
```
    System.out.println(
            "Charging can start."
    );

} else {

    System.out.println(
            "100 kW campus limit reached."
    );

    System.out.println(
            "Vehicle must join virtual queue."
    );
}

// Recommendation score
double currentLoad =
        getCurrentCampusLoad();

int recommendationScore =
        calculateRecommendationScore(
                true,
                true,
                currentLoad + chargerPower,
                true,
                currentBattery
        );
```

```
        System.out.println(
                "Recommendation Score = "
                        + recommendationScore
                        + "/100"
        );

        // Virtual queue score example
        int queuePriority =
                calculateQueuePriority(
                        10,
                        1,
                        30
                );

        System.out.println(
                "Queue Priority Score = "
                        + queuePriority
        );

        // CallableStatement demonstration
        displayStationUtilization();

        System.out.println(
                "\nSmartCharge Campus execution completed."
        );
    }
}
```

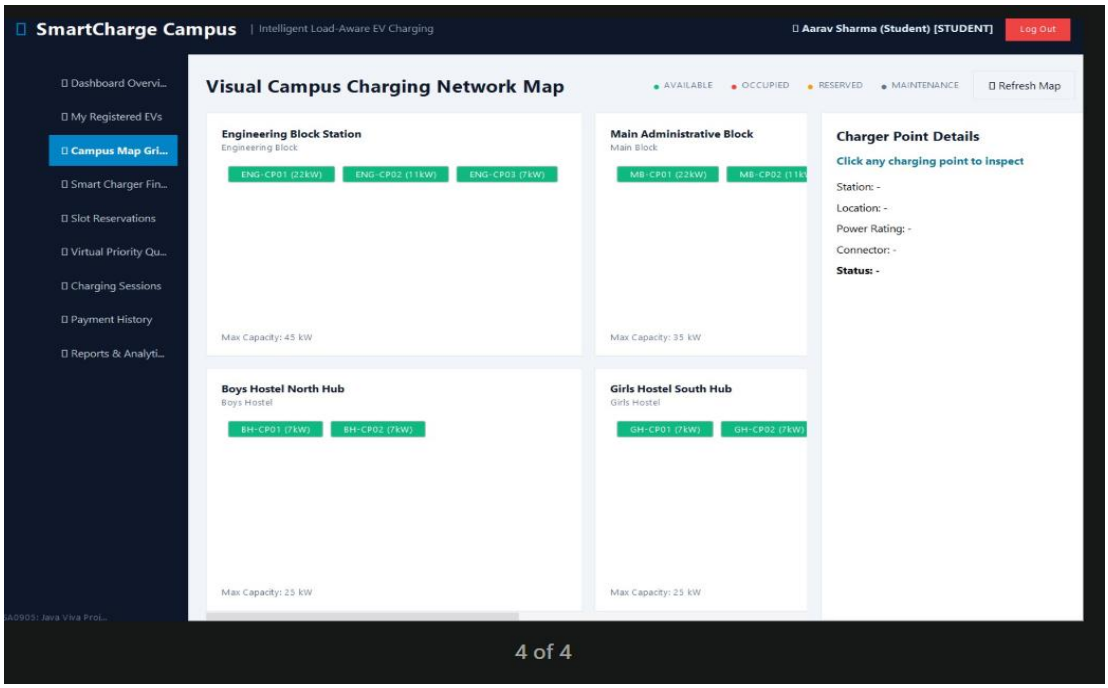
Output:

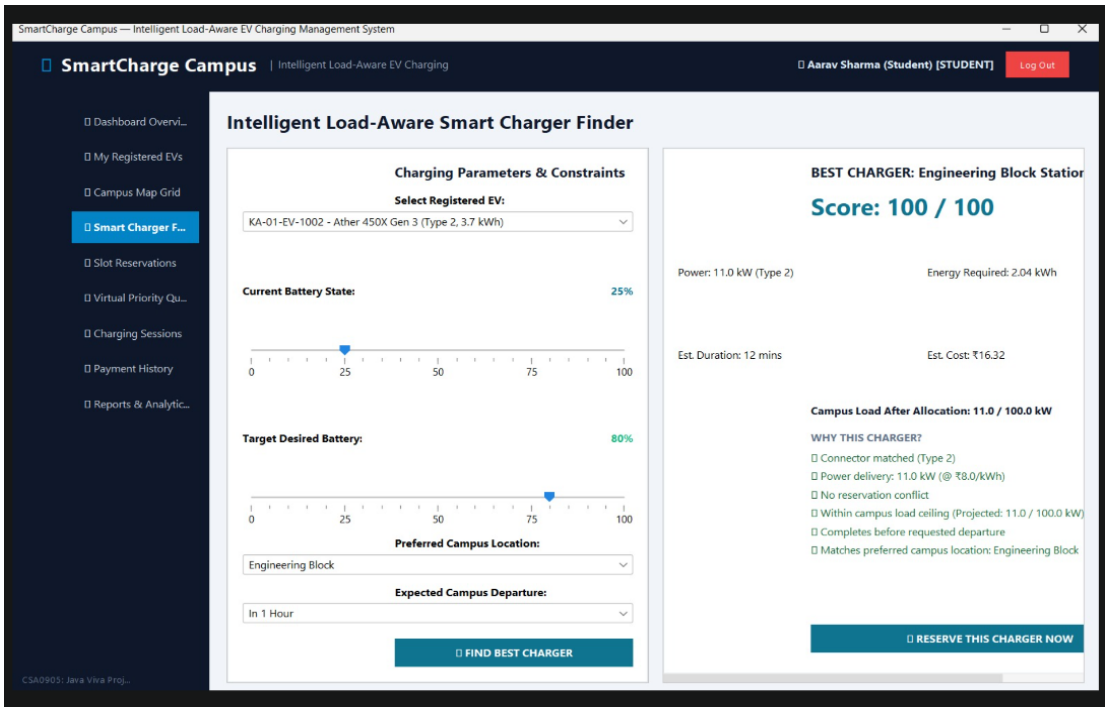
Student Dashboard:

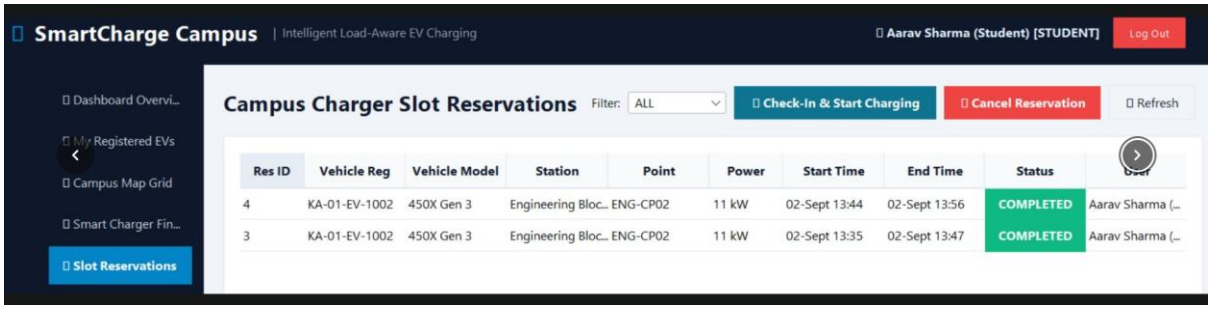


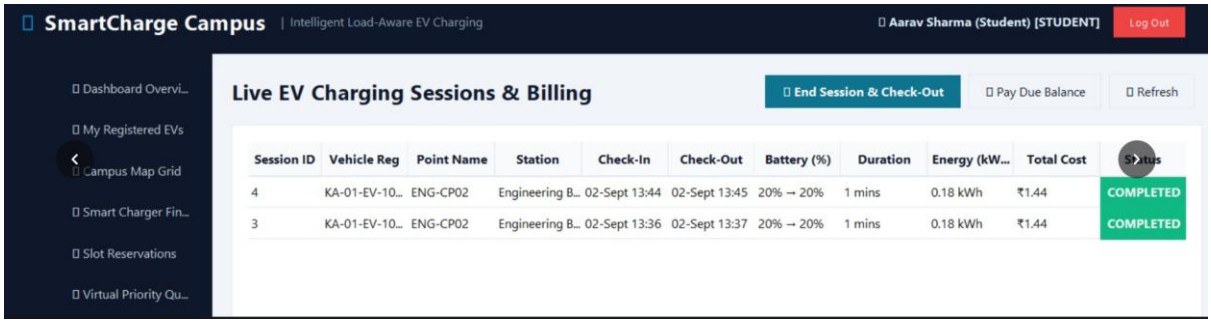


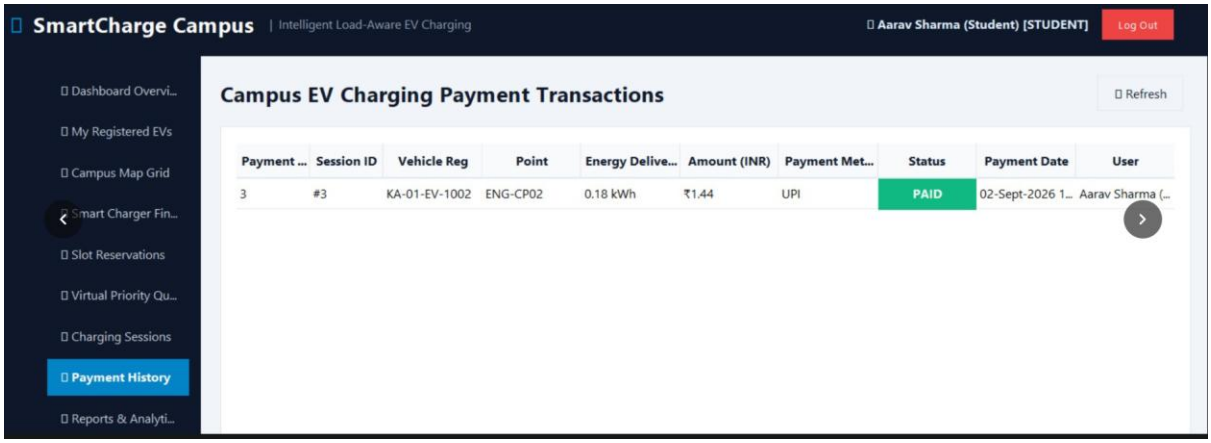

SmartCharge Campus | Intelligent Load-Aware EV Charging
Aarav Sharma (Student) [STUDENT]
Log Out
Dashboard Overvi...
My Registered EV...
Campus Map Grid
Smart Charger Fin...
Slot Reservations
Electric Vehicle Registry & CRUD Management
+ Register New Vehicle
Edit Vehicle
Delete
Refresh
ID
Registration No.
Manufacturer
Model
Battery (kWh)
Connector Type
Owner
2
KA-01-EV-1002
Ather
450X Gen 3
3.7 kWh
Type 2
Aarav Sharma (Student)
1
KA-01-EV-1001
Tata
Nexon EV MAX
40.5 kWh
CCS2
Aarav Sharma (Student)

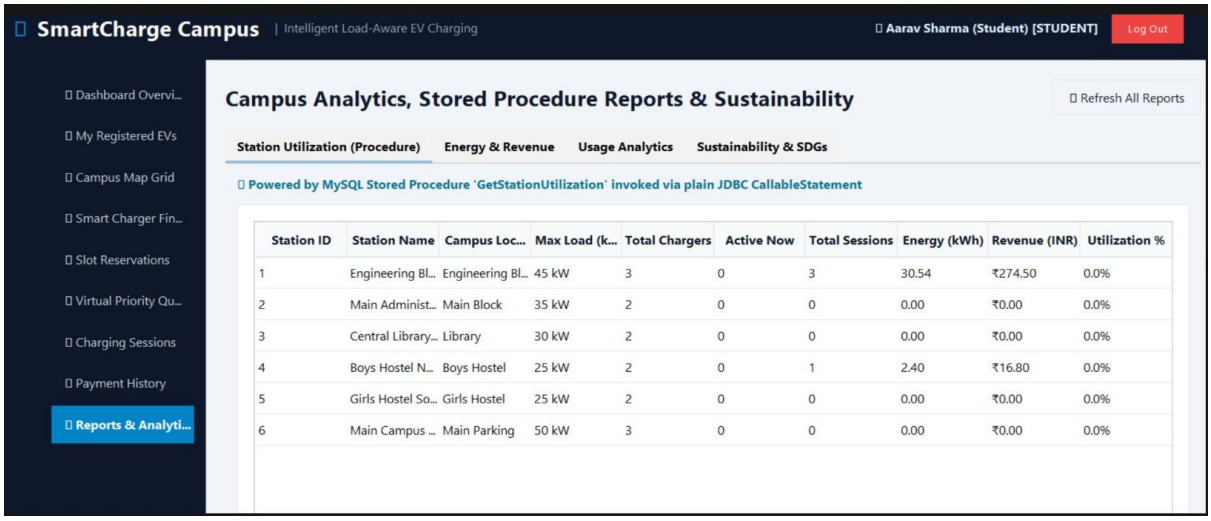

SmartCharge Campus | Intelligent Load-Aware EV Charging
Aarav Sharma (Student) [STUDENT]
Log Out
Dashboard Overvi...
My Registered EVs
Campus Map Gri...
Smart Charger Fin...
Slot Reservations
Virtual Priority Qu...
Charging Sessions
Payment History
Reports & Analyti...
Visual Campus Charging Network Map
AVAILABLE
OCCUPIED
RESERVED
MAINTENANCE
Refresh Map
Engineering Block Station
Engineering Block
ENG-CP01 (22kW)
ENG-CP02 (11kW)
ENG-CP03 (7kW)
Max Capacity: 45 kW
Main Administrative Block
Main Block
MB-CP01 (22kW)
MB-CP02 (11k
Max Capacity: 35 kW
Charger Point Details
Click any charging point to inspect
Station: -
Location: -
Power Rating: -
Connector: -
Status: -
Boys Hostel North Hub
Boys Hostel
BH-CP01 (7kW)
BH-CP02 (7kW)
Max Capacity: 25 kW
Girls Hostel South Hub
Girls Hostel
GH-CP01 (7kW)
GH-CP02 (7kW)
Max Capacity: 25 kW
CSA0905: Java Viva Proj...


SmartCharge Campus — Intelligent Load-Aware EV Charging Management System
SmartCharge Campus | Intelligent Load-Aware EV Charging
Aarav Sharma (Student) [STUDENT]
Log Out
Dashboard Overvi...
My Registered EVs
Campus Map Grid
Smart Charger F...
Slot Reservations
Virtual Priority Qu...
Charging Sessions
Payment History
Reports & Analytic...
Intelligent Load-Aware Smart Charger Finder
Charging Parameters & Constraints
Select Registered EV:
KA-01-EV-1002 - Ather 450X Gen 3 (Type 2, 3.7 kWh)
Current Battery State:
25%
0
25
50
75
100
Target Desired Battery:
80%
0
25
50
75
100
Preferred Campus Location:
Engineering Block
Expected Campus Departure:
In 1 Hour
FIND BEST CHARGER
BEST CHARGER: Engineering Block Station
Score: 100 / 100
Power: 11.0 kW (Type 2)
Energy Required: 2.04 kWh
Est. Duration: 12 mins
Est. Cost: ₹16.32
Campus Load After Allocation: 11.0 / 100.0 kW
WHY THIS CHARGER?
Connector matched (Type 2)
Power delivery: 11.0 kW (@ ₹8.0/kWh)
No reservation conflict
Within campus load ceiling (Projected: 11.0 / 100.0 kW)
Completes before requested departure
Matches preferred campus location: Engineering Block
RESERVE THIS CHARGER NOW
CSA0905: Java Viva Proj...

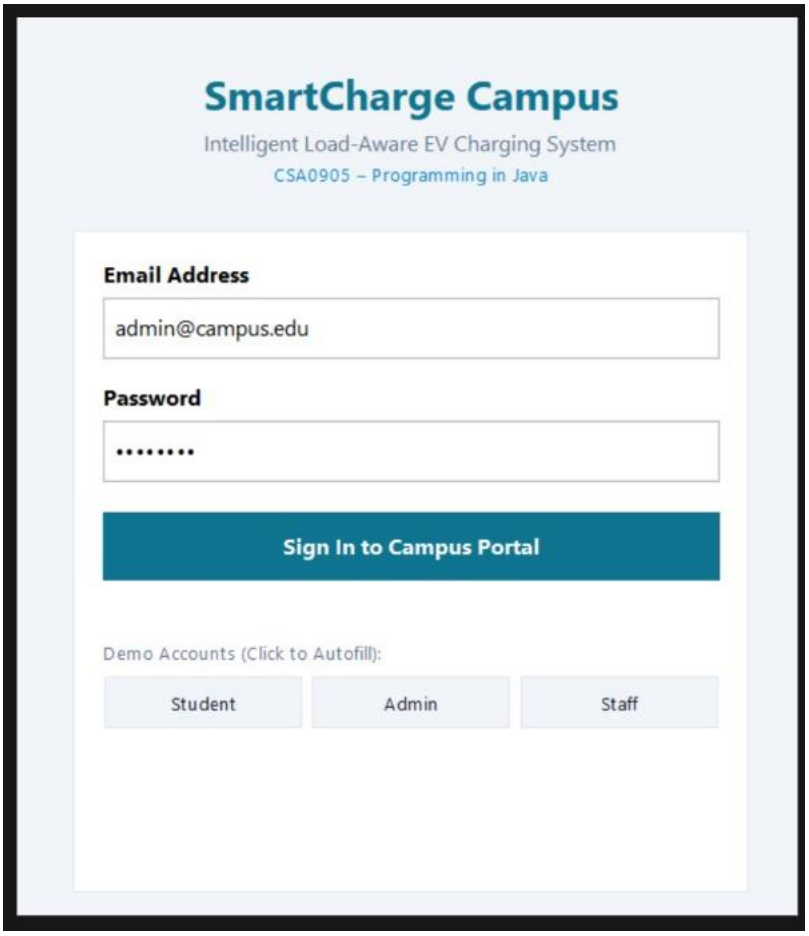

SmartCharge Campus
| Intelligent Load-Aware EV Charging
Aarav Sharma (Student) [STUDENT]
Log Out
Dashboard Overvi...
My Registered EVs
Campus Map Grid
Smart Charger Fin...
Slot Reservations
Campus Charger Slot Reservations
Filter: ALL
Check-In & Start Charging
Cancel Reservation
Refresh
Res ID | Vehicle Reg | Vehicle Model | Station | Point | Power | Start Time | End Time | Status | User
4 | KA-01-EV-1002 | 450X Gen 3 | Engineering Bloc... | ENG-CP02 | 11 kW | 02-Sept 13:44 | 02-Sept 13:56 | COMPLETED | Aarav Sharma (...
3 | KA-01-EV-1002 | 450X Gen 3 | Engineering Bloc... | ENG-CP02 | 11 kW | 02-Sept 13:35 | 02-Sept 13:47 | COMPLETED | Aarav Sharma (...

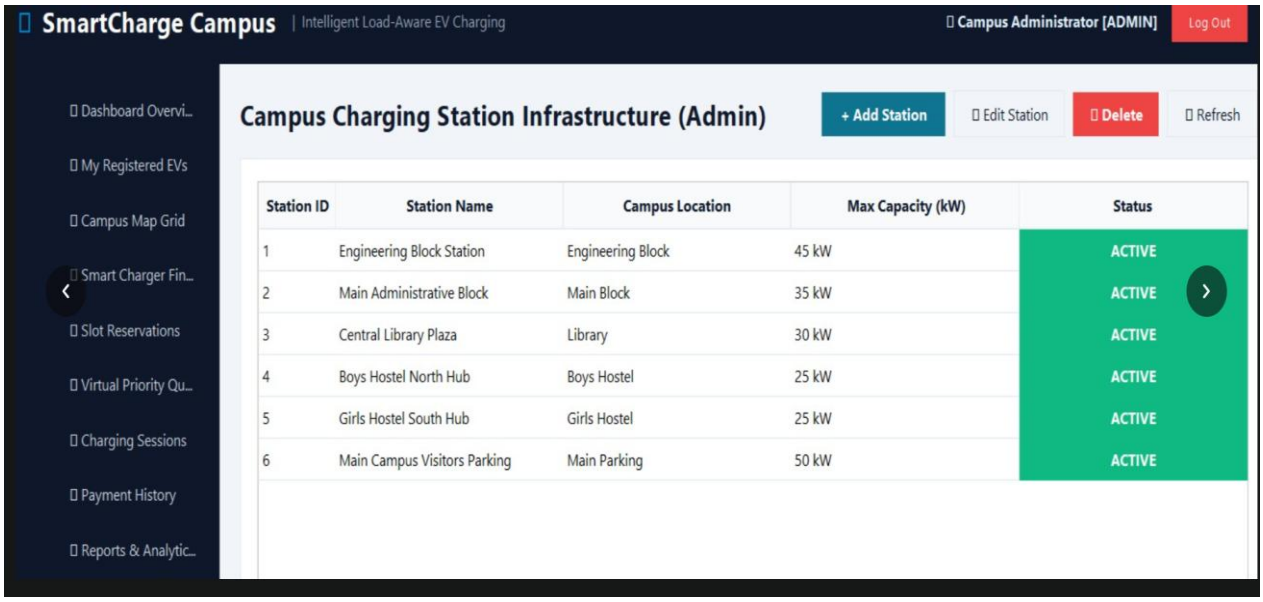

SmartCharge Campus
| Intelligent Load-Aware EV Charging
Aarav Sharma (Student) [STUDENT]
Log Out
Dashboard Overvi...
My Registered EVs
Campus Map Grid
Smart Charger Fin...
Slot Reservations
Virtual Priority Qu...
Live EV Charging Sessions & Billing
End Session & Check-Out
Pay Due Balance
Refresh
Session ID | Vehicle Reg | Point Name | Station | Check-In | Check-Out | Battery (%) | Duration | Energy (kW... | Total Cost | Status
4 | KA-01-EV-10... | ENG-CP02 | Engineering B... | 02-Sept 13:44 | 02-Sept 13:45 | 20% → 20% | 1 mins | 0.18 kWh | ₹1.44 | COMPLETED
3 | KA-01-EV-10... | ENG-CP02 | Engineering B... | 02-Sept 13:36 | 02-Sept 13:37 | 20% → 20% | 1 mins | 0.18 kWh | ₹1.44 | COMPLETED

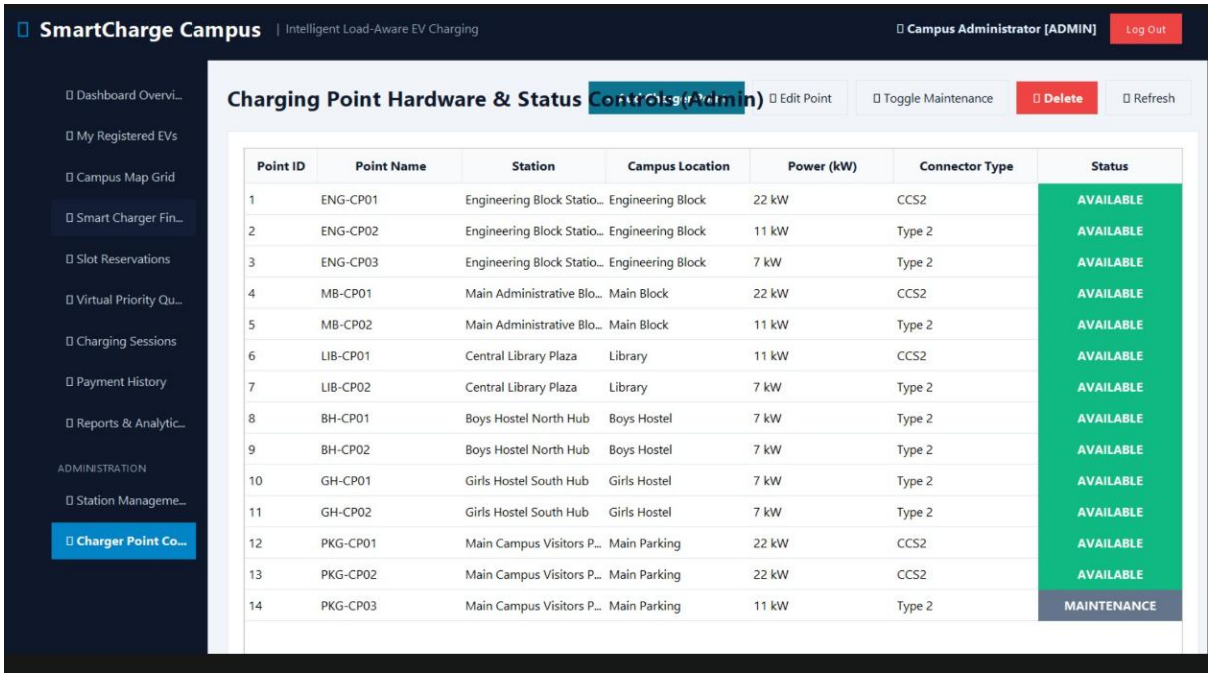

SmartCharge Campus
| Intelligent Load-Aware EV Charging
Aarav Sharma (Student) [STUDENT]
Log Out
Dashboard Overvi...
My Registered EVs
Campus Map Grid
Smart Charger Fin...
Slot Reservations
Virtual Priority Qu...
Charging Sessions
Payment History
Reports & Analyti...
Campus EV Charging Payment Transactions
Refresh
Payment ... | Session ID | Vehicle Reg | Point | Energy Delive... | Amount (INR) | Payment Met... | Status | Payment Date | User
3 | #3 | KA-01-EV-1002 | ENG-CP02 | 0.18 kWh | ₹1.44 | UPI | PAID | 02-Sept-2026 1... | Aarav Sharma (...


SmartCharge Campus
| Intelligent Load-Aware EV Charging
Aarav Sharma (Student) [STUDENT]
Log Out
Dashboard Overvi...
My Registered EVs
Campus Map Grid
Smart Charger Fin...
Slot Reservations
Virtual Priority Qu...
Charging Sessions
Payment History
Reports & Analyti...
Campus Analytics, Stored Procedure Reports & Sustainability
Refresh All Reports
Station Utilization (Procedure)
Energy & Revenue
Usage Analytics
Sustainability & SDGs
Powered by MySQL Stored Procedure 'GetStationUtilization' invoked via plain JDBC CallableStatement
Station ID | Station Name | Campus Loc... | Max Load (k... | Total Chargers | Active Now | Total Sessions | Energy (kWh) | Revenue (INR) | Utilization %
1 | Engineering Bl... | Engineering Bl... | 45 kW | 3 | 0 | 3 | 30.54 | ₹274.50 | 0.0%
2 | Main Administ... | Main Block | 35 kW | 2 | 0 | 0 | 0.00 | ₹0.00 | 0.0%
3 | Central Library... | Library | 30 kW | 2 | 0 | 0 | 0.00 | ₹0.00 | 0.0%
4 | Boys Hostel N... | Boys Hostel | 25 kW | 2 | 0 | 1 | 2.40 | ₹16.80 | 0.0%
5 | Girls Hostel So... | Girls Hostel | 25 kW | 2 | 0 | 0 | 0.00 | ₹0.00 | 0.0%
6 | Main Campus ... | Main Parking | 50 kW | 3 | 0 | 0 | 0.00 | ₹0.00 | 0.0%

Admin Dashboard:






SmartCharge Campus
| Intelligent Load-Aware EV Charging
Campus Administrator [ADMIN]
Log Out
Dashboard Overvi...
My Registered EVs
Campus Map Grid
Smart Charger Fin...
Slot Reservations
Virtual Priority Qu...
Charging Sessions
Payment History
Reports & Analytic...
ADMINISTRATION
Station Manageme...
Charger Point Co...
Charging Point Hardware & Status Controls (Admin)
Edit Point
Toggle Maintenance
Delete
Refresh
Point ID
Point Name
Station
Campus Location
Power (kW)
Connector Type
Status
1 ENG-CP01 Engineering Block Statio... Engineering Block 22 kW CCS2 AVAILABLE
2 ENG-CP02 Engineering Block Statio... Engineering Block 11 kW Type 2 AVAILABLE
3 ENG-CP03 Engineering Block Statio... Engineering Block 7 kW Type 2 AVAILABLE
4 MB-CP01 Main Administrative Blo... Main Block 22 kW CCS2 AVAILABLE
5 MB-CP02 Main Administrative Blo... Main Block 11 kW Type 2 AVAILABLE
6 LIB-CP01 Central Library Plaza Library 11 kW CCS2 AVAILABLE
7 LIB-CP02 Central Library Plaza Library 7 kW Type 2 AVAILABLE
8 BH-CP01 Boys Hostel North Hub Boys Hostel 7 kW Type 2 AVAILABLE
9 BH-CP02 Boys Hostel North Hub Boys Hostel 7 kW Type 2 AVAILABLE
10 GH-CP01 Girls Hostel South Hub Girls Hostel 7 kW Type 2 AVAILABLE
11 GH-CP02 Girls Hostel South Hub Girls Hostel 7 kW Type 2 AVAILABLE
12 PKG-CP01 Main Campus Visitors P... Main Parking 22 kW CCS2 AVAILABLE
13 PKG-CP02 Main Campus Visitors P... Main Parking 22 kW CCS2 AVAILABLE
14 PKG-CP03 Main Campus Visitors P... Main Parking 11 kW Type 2 MAINTENANCE

Student 1 –V. Bala Siva Prasad Reddy(192424316)

GUI Design and User Interface

Contributed primarily to the Graphical User Interface (GUI), event handling, input validation and user interaction of the E-Vehicle Charging Management System. The work involved designing the overall structure of the application interface using Java AWT/Swing components. The required controls such as labels, text fields, buttons, text areas and menus were organized using suitable layout managers to make the application simple, systematic and user-friendly.The GUI was designed to provide access to the major functions of the charging management system, including user registration, vehicle registration, charging-station and charging-point management, reservation, charging-session management and viewing records. Event handling was implemented for the different buttons so that the corresponding database or application operation could be performed when the user interacts with the interface.

Input validation was also considered to ensure that required fields were not left empty and that values entered into fields such as vehicle information, battery capacity, energy consumption and other numerical fields were appropriate. Error messages were displayed wherever necessary to make the system easier to operate and to prevent incorrect data from being submitted.The GUI was tested with different inputs and different button operations to verify that the interface responds correctly. The member also contributed to integrating the GUI with the application logic and JDBC layer so that information entered through the interface could be processed and stored in the database. This contribution directly supports the assignment requirements related to GUI controls, layout managers, event-driven programming and validation.

Major Contribution:

- Designed AWT/Swing GUI.
- Used labels, text fields, buttons, text areas and menus.
- Implemented layout managers.
- Implemented event handling.
- Designed user interaction and navigation.
- Tested GUI functionality.

Student 2-S.Mohamed Imraan(192424001)

MySQL Database, JDBC and SQL Operations

Contributed primarily to the MySQL database design, JDBC connectivity and SQL operations of the E-Vehicle Charging Management System. The work involved designing a relational database capable of storing the information required by the charging-management application. Tables were created for important entities including users, vehicles, charging stations, charging points, reservations, charging sessions and payments, as specified for the system. Primary keys were defined to uniquely identify records, while foreign-key relationships were established between related tables to maintain data consistency. The database structure was tested using MySQL Workbench to ensure that the tables and relationships were created correctly.

The member was also responsible for establishing JDBC connectivity between the Java application and MySQL. The MySQL JDBC driver, database URL, username and password were configured so that the Java application could communicate with the database. SQL operations required by the application were implemented through JDBC, including INSERT, SELECT, UPDATE and DELETE operations.

Major responsibilities

- Designed the MySQL database structure.
- Created tables for users and vehicles.
- Created charging station and charging point tables.
- Created reservation and charging-session tables.
- Created payment records table.
- Defined primary and foreign key relationships.
- Established JDBC connectivity with MySQL.
- Implemented INSERT operations.
- Implemented SELECT operations.
- Implemented UPDATE operations.
- Implemented DELETE operations.
- Tested database records using MySQL Workbench.
- Handled database connection and SQL exceptions.

Student 3-Allen Gabriel(192421089)

Charging Logic, Reservation, Cost Calculation, Testing and Documentation

Contributed primarily to the core E-Vehicle charging-management functionality, reservation process, charging-session management, cost calculation, testing and project documentation. The work focused on implementing the logical operations required to manage the charging process from selecting a charging station through completion of a charging session.The member worked on the logic for checking charging-point availability and managing charging-point status. The system was designed to distinguish between available and unavailable charging points so that a charging point could not be incorrectly assigned when it was already occupied or reserved. Reservation functionality was implemented to associate a vehicle with an appropriate charging point and reservation period.

The charging-session functionality was developed to record information such as the selected vehicle, charging point, start time, end time and energy consumed. The member also worked on calculating the charging cost based on the energy consumed and the applicable tariff. Payment-related information was incorporated so that the amount associated with a completed charging session could be recorded.The member carried out functional testing of the complete application after the GUI, Java logic and database components were integrated. Test cases included valid user and vehicle registration, charging-point availability, successful and unsuccessful reservations, charging-session operations, energy-consumption entry, cost calculation, database retrieval and invalid input. Errors identified during testing were analyzed and corrected in coordination with the other team members.

Major responsibilities

- Implemented charging-point availability logic.
- Implemented charging-point reservation logic.
- Managed charging-point status.
- Implemented charging-session workflow.
- Recorded charging duration.
- Recorded energy consumption.